Lecturer:
Prof. Dr. Christian Tschudin (christian.tschudin@unibas.ch)

Teaching Assistant:
Ali Ajorian (ali.ajorian@unibas.ch)

Tutors:
Mark Starzynski (mark.starzynski@unibas.ch)
Sebastian Lenzlinger (sebastian.lenzlinger@unibas.ch)

Uploaded on
Wed., 17. April 2024

Deadline
Sun., 12. May 2024 (23:55)
Upload to ADAM

Modalities

The exercises have to be done in **groups of 2 students**. Upload only the final version of the exercise to Adam and specify your group partner.

Please upload your short report, containing all answers to the asked questions and pointers to code files as a .pdf file as well as the source files for your implementation as one single ZIP file to ADAM by the deadline at the latest.

Introduction

In this exercise, you will extend your chat application from the previous two exercises with additional functionalities based on state-based Conflict-Free Replicated Data Types (CRDTs). As presented in class, you will document your design by specifying:

(a) **partial order**: all possible states and their partial order;

(b) **merge**: the function used to combine two states, e.g. current state and received state;

(c) **monotonic operations**: all other operations that modify the state and argue they all result in a newer state that is either equal or larger than the previous state.

Task 1 - UDP-Broadcast: Last Activity Status (3 Points)

Implement a standalone utility that tracks when was the last time any participant has received a packet from the other participants. Your utility should track activity based on self-assigned user names and the last Unix timestamp at which a packet from that participant was last received. Each replica should periodically broadcast, e.g. every 10 seconds, its current local state.
For example, suppose there are three participants, Alice, Bob, and Carol, each operating one replica and Alice has last received from:

- **Alice**: `Last activity for Alice (16h40), Bob (17h00)`
- **Bob**: `Last activity for Alice (16h58) and Bob (17h01)`
- **Carol**: `Last activity for Alice (16h50), Bob (17h05) and Carol (17h03)`

Alice's utility should display: `Last activity for Alice (16h58), Bob (17h05), and Carol (17h03)`. This indication should constantly be updated after receiving new updates.
Assume all participants' user names are unique, that clocks for each participant computers are synchronized within a few seconds so that local clocks are mostly similar, and that participants only broadcast under their own user names and do not try to impersonate others.

(a) Describe your state-based CRDT design according to its partial order, merge, and monotonic operations.

(b) Describe how the state of your CRDT is encoded in a single UDP packet with a maximum size of 576 bytes[1] and the associated limitations on the number of participants and the size of participants' user names.

(c) Implement your utility and test with at least 2 machines to verify that all replicas are periodically consistent, i.e. they periodically show the same last recorded activity for all participants, even if one of the replicas is temporarily offline.

[1] See `https://stackoverflow.com/questions/1098897/what-is-the-largest-safe-udp-packet-size-on-the-internet`.

Task 2 - UDP-Broadcast: Frontier Computation (3 points)

In this exercise, you will implement a UDP-based mechanism for the Git-based chat application you implemented in Exercise 2, to compute how many messages are missing locally. This mechanism should enable each replica to:

- Periodically broadcast their *local frontier*, i.e. the latest messages they have already replicated (hint: use *vector clocks*);
- Compute the *global frontier*, i.e. the latest messages replicated by any replica, to provide a user indication when the local frontier and the global frontier differ and by how many messages from each participant;
- Track the current replication state of other replicas by updating the corresponding `refs/remotes` references based on the local frontiers directly received from them.

(a) Describe your frontier design as a state-based CRDT according to its partial order and merge operations. Describe how the local frontier is computed from the local message graph, and argue why adding a new message to the local message graph is a monotonic operation;

(b) Describe how the local frontier is encoded in a single UDP packet and the associated limitations on the number of participants, the size of participants' user names, and the number of messages per participant of your implementation;

(c) Implement your frontier protocol and test with at least 2 machines to verify that all replicas are eventually consistent, i.e. compute the same global frontiers, when messages are added locally and independently in each replica, even if one of the replicas is temporarily offline.

Task 3 - UDP-Broadcast: Replication of Git-based Chat (4 Points)

In this exercise, you will implement a UDP-based mechanism for the Git-based chat application you implemented in Exercise 2, that will replace the push and pull commands you previously used to propagate updates.
This process should broadcast one random local message that is missing in another replica (if any) after receiving the *local frontier* of the other replica. The message chosen can be from any participant but should be the next missing message for that participant, based on the local frontier received.
Upon reception of a UDP broadcast with a message, the other replica should add the message locally (if still missing). Then, if all messages up to the new message have been successfully replicated already, the `refs/heads/<PARTICIPANT>` reference for that message author should be updated.
Also, upon reception of the `local frontier` of another replica, the
`refs/remotes/<ORIGIN>/<PARTICIPANT>` references should also be updated, to follow the behaviour of the usual `push` and `pull` reconciliation operations.

(a) Describe how a single message, stored locally as a Git commit, is encoded as a UDP packet that can be distinguished from Task 2 - and the associated limitations on the number of participants, the size of participants' user names, and the size of the content of a message;

(b) Implement your reconciliation protocol and test with at least 3 machines to verify that all replicas are eventually consistent, i.e. they replicate all the same messages and compute the same local frontiers, even if one of the replicas is temporarily offline, and two replicas are never online at the same time.

(c) Implement and verify that the `refs/remotes` references are correctly updated.

(d) Imagine a practical scenario with several users in the chatroom, where one user has been offline for some time and comes back online after a while. What could be potential issues with your implementation and can you provide possible solutions for those issues?